Universidad Autónoma Gabriel René Moreno
Facultad de Ingeniería en Ciencias de la Computación y Telecomunicaciones



INFORME DEL PROYECTO

Conducción en tercera persona · Java, Maven, LWJGL y OpenGL 3.3

Materia: Programación Grafica

Docente: Kenji Kawaida Villegas

Estudiante: Ian Patrick Valdez Farfan

Fecha de entrega: 10/10/2026

Santa Cruz de la Sierra – Bolivia

1. Descripción general

El proyecto es una simulación en la que el usuario conduce un automóvil en tercera persona por una ciudad 3D para completar cinco entregas. Partiendo del proyecto de referencia de la materia (ciudad de 7 × 7 celdas con conducción básica), se amplió la ciudad a 11 × 11 celdas y se incorporaron iluminación día/noche, farolas y faros con cálculo de luz, decoración urbana, semáforos animados, minimapa, reinicio y, como mejoras opcionales, tráfico autónomo, un panel de información dentro de la ventana y letreros de sector.

Aspecto	Detalle
Lenguaje y construcción	Java (compilado para Java 17; probado con JDK 25), Maven
Gráficos	LWJGL 3.3.6 (GLFW + OpenGL), OpenGL 3.3 Core Profile, GLSL 330 core
Clase principal	`com.graphics.AppConduccion`
Plataforma	Windows x64 (librerías nativas `natives-windows`)

2. Instrucciones de ejecución

Requisitos: JDK 17 o superior, Apache Maven, y una tarjeta gráfica con driver compatible con OpenGL 3.3.

1. Abrir una terminal en la carpeta del proyecto que contiene el archivo `pom.xml`.
2. Ejecutar la aplicación: `mvn compile exec:exec`
3. Ejecutar las pruebas automáticas (opcional): `mvn test`
4. Desde VS Code también se puede usar Run and Debug → "Ciudad Conduccion (AppConduccion)", o el botón Run sobre el método `main` de `AppConduccion.java`.

Las advertencias `sun.misc.Unsafe` que aparecen en la consola con Java 25 provienen de LWJGL y no afectan el funcionamiento.

3. Diseño de la ciudad

La ciudad se define como una matriz de caracteres de 11 × 11 (`MAPA_BASE` en `MapaCiudad.java`). Cada celda mide 10 × 10 unidades, igual que en el mapa original, por lo que la ciudad ocupa 110 × 110 unidades con límites en ±55. Las filas corresponden al eje Z (fila 0 = norte) y las columnas al eje X (columna 0 = oeste).

El diseño es **data-driven**: el suelo, la barrera del borde, los límites de circulación, las colisiones, las farolas, los semáforos, los destinos, la distancia de la cámara panorámica y la escala del minimapa se calculan a partir de la misma matriz. Así, ampliar o modificar la ciudad solo requiere editar el mapa.

Sectores

El cuadrante noroeste (filas y columnas 0 a 6) conserva la disposición de manzanas del mapa original de 7 × 7, al que se le agregaron farolas, semáforos y dos destinos. Los otros tres sectores son nuevos y están conectados por la red vial.

Sector	Filas	Columnas	Origen
CENTRO HISTÓRICO	0 – 6	0 – 6	Mapa original 7 × 7 ampliado
DISTRITO ESTE	0 – 6	7 – 10	Nuevo
BARRIO SUR	7 – 10	0 – 6	Nuevo (contiene la salida)
ZONA SURESTE	7 – 10	7 – 10	Nuevo

Destinos de entrega

N.º	Nombre	Fila, columna	Sector
1	Panadería	0, 9	Distrito Este
2	Hospital	5, 10	Distrito Este
3	Escuela	10, 7	Zona Sureste
4	Biblioteca	6, 3	Centro Histórico
5	Estación	3, 0	Centro Histórico
S	Salida (mirando al norte)	9, 0	Barrio Sur

Resumen de elementos

Elemento	Implementado	Mínimo exigido
Tamaño del mapa	11 × 11	11 × 11
Manzanas con edificio	19	12
Parques	6	4
Farolas (9 en sectores nuevos)	14	9
Cruces con semáforo	7	—
Destinos de entrega	5	—

Las alturas de los edificios (6 a 18 unidades) y sus colores (paleta de 7) se obtienen de una función de la fila y la columna, por lo que cada edificio conserva siempre su apariencia; algunos incluyen una torre superior. Una prueba automática verifica, mediante búsqueda en anchura desde la salida, que todos los destinos son accesibles por calles.

4. Controles

Tecla	Acción
W / ↑	Acelerar
S / ↓	Frenar y reversa
A, D / ←, →	Girar
Espacio	Freno fuerte

Tecla	Acción
C	Cambiar cámara: tercera persona → aérea → panorámica
Q / E	Girar la vista panorámica
N	Alternar día / noche
F	Encender / apagar faros
M	Mostrar / ocultar minimapa
R	Reiniciar el recorrido
H	Mostrar / ocultar ayuda
P	Pausa
ESC	Salir

5. Funciones implementadas

5.1 Conducción y escenario

- **Física del auto** con aceleración, resistencia exponencial, freno, reversa y giro proporcional a la velocidad; todo el movimiento usa el tiempo entre cuadros (deltaTime, limitado a 50 ms).
- **Colisiones** círculo–rectángulo (radio 1,65) contra las manzanas y los límites del mapa. Al chocar, el auto se desliza a lo largo de la pared en lugar de detenerse en seco.
- **Tres cámaras** (tercera persona, aérea y panorámica giratoria) con transición suave.
- **Entregas:** cinco destinos con marca dorada y baliza; una entrega se completa al llegar y detenerse. Al terminar se muestra la pantalla de victoria con el tiempo.
- **Indicador en el título** de la ventana, actualizado cuatro veces por segundo: velocidad, día/noche, faros, cámara, entregas, destino y tiempo.

5.2 Iluminación

- **Día y noche (N):** luz ambiental más una luz direccional (sol o luna) con cambios visibles en el cielo y en las superficies; de noche la iluminación permite conducir.
- **Luz difusa de Lambert:** cada cara del cubo tiene su normal y recibe `max(dot(N, L), 0)`, por lo que cada cara de un edificio se ilumina según su orientación.
- **Farolas:** 14 luces puntuales calculadas en el fragment shader, con atenuación `1 / (1 + 0,09·d + 0,016·d²)` y alcance de 20 unidades. Iluminan el suelo y los edificios cercanos.
- **Emisión frente a iluminación:** la bombilla se dibuja con color emisivo (no se oscurece), pero eso no ilumina el entorno; la luz que recibe el entorno proviene del cálculo de luz puntual.
- **Faros (F):** dos focos que siguen la posición y orientación del auto, con cono de unos 28° (suavizado con smoothstep), atenuación y alcance de 30 unidades.

5.3 Funciones urbanas

- **Decoración:** parques con árboles, bancos, fuente, senderos y basureros; edificios con ventanas (de noche se encienden cerca del 70 %); pasos peatonales en las calles que llegan a cada cruce.

- **Semáforos animados:** ciclo por tiempo verde (5 s) → amarillo (2 s) → rojo (7 s); los dos ejes de cada cruce están desfasados para que nunca estén ambos en verde.
- **Minimapa (M):** vista superior de toda la ciudad con el norte arriba, la posición y orientación del auto, el destino activo y el tráfico. Se dibuja en una segunda pasada con `glViewport`, `glScissor` y proyección ortográfica propios, restaurando el estado al terminar para no alterar la vista principal.
- **Redimensionamiento:** el callback de tamaño del framebuffer actualiza la vista principal y recalcula el minimapa (30 % del lado menor, anclado a la esquina superior derecha).
- **Reinicio (R):** restablece la posición, orientación y velocidad del auto, además de las entregas, el tiempo, el tráfico y los semáforos, permitiendo repetir toda la actividad.

5.4 Mejoras opcionales

- **Tráfico autónomo:** cinco vehículos que eligen su ruta celda por celda, circulan por el carril derecho, giran con curvas Bézier, respetan los semáforos, mantienen distancia con el vehículo de adelante y no atraviesan al jugador.
- **Interfaz dentro de la ventana:** panel con velocidad (con barra), estado de luces, cámara, sector y destino con distancia y rumbo; ayuda de controles (H), menú de pausa (P), aviso de entrega y pantalla de victoria. Usa un shader propio y una fuente de píxeles 5 × 7 generada por código.
- **Mejora urbana:** letreros 3D con el nombre de cada sector, que se iluminan de noche.

6. Organización del código

La clase `AppConduccion` coordina la ventana, la entrada y el ciclo principal. El resto del código está en `com.graphics.juego`, separado por responsabilidad:

Paquete	Contenido
`motor`	`Renderizador` (shaders 3D, cubo unitario con VAO/VBO), `Matriz` (perspectiva, ortográfica, lookAt), `Fuente`
`ciudad`	`MapaCiudad` (matriz, sectores, colisiones, farolas, destinos), `DibujoCiudad`, `Letreros`
`conduccion`	`Auto`, `ControlesAuto`, `Obstaculos`, `ModeloAuto`, `Camara`
`iluminacion`	`Iluminacion` (día/noche, farolas, faros)
`funciones`	`Entregas`, `Semaforos`, `Minimapa`, `Hud`, `PanelHud`
`trafico`	`VehiculoAutonomo`, `Trafico`

Toda la escena se construye a partir de un único cubo de lado 1 que el vertex shader escala, rota y traslada. Los shaders están escritos dentro del código Java (GLSL 330 core).

Pruebas automáticas

`ConduccionTest.java` contiene 10 pruebas de la lógica del juego (`mvn test`):

Prueba	Qué verifica
`testMapaAmpliado`	Mapa ≥ 11 × 11, ≥ 12 edificios, ≥ 4 parques, ≥ 9 farolas
`testDestinosAlcanzablesPorCalles`	Todos los destinos son accesibles desde la salida (BFS)

Prueba	Qué verifica
`testCallesNuevasTransitables`	Las calles de los sectores nuevos son transitables
`testObstaculosYLimites`	No se puede entrar a edificios, parques, aceras ni salir del mapa
`testChoqueNoAtraviesa`	Chocar 10 s contra una manzana nunca deja al auto dentro
`testMovimientoIndependienteDeLosFps`	El mismo recorrido a 30 y 144 FPS termina en el mismo lugar
`testTraficoSigueLasCalles`	5 min simulados: los vehículos no pisan aceras ni salen del mapa, y giran
`testSemaforoSecuencia`	Secuencia verde → amarillo → rojo; nunca dos ejes en verde
`testEntregasCompletasYReinicio`	Las entregas cuentan solo al llegar y frenar; tras reiniciar se repiten
`testReinicio`	R devuelve posición, ángulo y velocidad iniciales

7. Limitaciones

- **Sin sombras:** la luz de las farolas y los faros atraviesa los edificios. Solo hay luz ambiental y difusa (sin brillo especular). El cambio día/noche es instantáneo.
- **Luces:** el shader admite hasta 16 farolas. Los faros de los vehículos autónomos y las ventanas encendidas son solo visuales (no iluminan el entorno).
- **Colisiones simplificadas:** el auto es un círculo y cada manzana un rectángulo completo; árboles, postes y bancos no tienen colisión propia. No hay respuesta física real (rebotes, daños).
- **Cámara:** no tiene colisión; en tercera persona puede quedar dentro de un edificio si el auto retrocede contra una pared.
- **Tráfico:** rutas aleatorias con velocidad fija; en cruces sin semáforo no hay reglas de prioridad, y si el jugador se detiene en su carril el vehículo espera indefinidamente.
- **Semáforos:** son solo visuales para el jugador (no lo detienen).
- **Mapa:** celdas fijas de 10 × 10, una manzana por celda, una sola salida, máximo 9 destinos y orden de entregas fijo.
- **Interfaz:** la fuente solo tiene mayúsculas sin tildes.
- **Rendimiento:** cada caja es una llamada de dibujo (sin instancing). Por debajo de 20 FPS el juego corre más lento que el tiempo real por el límite de deltaTime.
- **Plataforma:** el pom.xml incluye solo las librerías nativas de Windows; en macOS o Linux se deben agregar las correspondientes (y en macOS, `-XstartOnFirstThread`).

8. Recursos y fragmentos externos

8.1 Proyecto de referencia de la materia

El trabajo parte del proyecto de referencia entregado por el docente. `App.java`, `AppMovimientoTeclado.java` y `AppZoom.java` son ejemplos del docente y no se modificaron; `AppCiudad.java` (ciudad 7 × 7) se completó y corrigió. Del proyecto de referencia se conservaron o adaptaron los siguientes elementos:

Origen	Elementos conservados o adaptados
clase1	Estructura del ciclo principal; límite de deltaTime de 50 ms; dibujo con un cubo unitario de 36 vértices con normales; funciones `caja()` / `cajaGirada()`; matriz de celdas y marcas viales (convertida a letras y ampliada a 11 × 11); edificio con acera, cuerpo y cornisa
clase2	Física del auto (aceleración, resistencia, freno, reversa, giro); convención de ángulo; modelo 3D del auto; colisión círculo–rectángulo; cámara de seguimiento con tecla C; título con velocidad y tecla R
clase3	Estructura del fragment shader (ambiente, sol con Lambert, farolas con atenuación, faros con smoothstep); idea de emisión para bombillas; teclas N y F
clase4	Regla de entregas, marca dorada con baliza y mensaje de victoria; ideas de decoración (árboles, bancos, ventanas nocturnas, pasos peatonales, semáforo de 3 luces); minimapa con `glScissor` y `glViewport`
ConduccionTest	Estructura de las 3 pruebas originales, adaptadas al mapa 11 × 11; se agregaron 7 pruebas

Elementos nuevos sin equivalente en la referencia: mapa por letras con sectores, clase de matrices con proyección ortográfica real, tráfico autónomo, HUD con shader y fuente propios, letreros 3D, pausa y ayuda, semáforos con ejes desfasados, parques con fuente y senderos, edificios con torre, tres cámaras con suavizado y redibujo durante el redimensionado.

8.2 Librerías

Librería	Uso
LWJGL 3.3.6 (lwjgl, lwjgl-glfw, lwjgl-opengl)	Enlace de Java con las bibliotecas nativas
GLFW (incluido en LWJGL)	Ventana, teclado y callbacks
OpenGL 3.3 Core	Renderizado (provisto por el driver de la GPU)
JUnit 3.8.1	Pruebas unitarias
Maven (compiler 3.13.0, exec 3.1.0)	Compilación y ejecución

8.3 Técnicas y fórmulas conocidas

No se copió código de sitios web o tutoriales. Se emplearon técnicas estándar de computación gráfica, como las que describe LearnOpenGL ("Basic Lighting", "Light casters") y la documentación de LWJGL/GLFW:

- Iluminación difusa de Lambert; atenuación de luz puntual `1 / (1 + kl·d + kq·d²)`; focos con smoothstep sobre el coseno del ángulo; niebla por distancia.
- Matrices de perspectiva, ortográfica y lookAt (fórmulas estándar de gluPerspective, glOrtho y GLM).
- Integración con deltaTime (Euler explícito), resistencia exponencial y suavizado de cámara `1 − e^(−k·dt)`.
- Colisión círculo–rectángulo por punto más cercano; curvas Bézier cuadráticas y cúbicas para los giros del tráfico; búsqueda en anchura (BFS) en las pruebas.
- Funciones hash para alturas, colores y ventanas: constantes de spatial hashing (Teschner et al., 2003) y de MurmurHash2.

8.4 Recursos gráficos

No se utilizaron texturas, modelos, imágenes ni fuentes externas. Toda la geometría se construye con cajas a partir de un único cubo, y la única textura (el atlas de la fuente 5 × 7 del HUD) se genera en memoria a partir de datos definidos en `Fuente.java`.